

Evaluating LLMs

How we know it's better · video 2:37

This lesson answers one question: after changing a prompt, a model or its precision, how do you know the result is **better**? You need an **evaluation**: questions with known answers and an automatic **check**, run after every change. A real test of three models shows the traps: a check that is too strict, a test that is too easy, and scores that hide wrong reasoning, which you only find by **reading the outputs**.

How to use this guide: master one concept before moving to the next.

1. Watch the whole video once, without stopping.
2. Then take the concepts **in order**. For each one: read the short explanation, look at the picture from the video (▶ is the time in the video), and answer every question in its *Check* without looking anything up.
3. Check your answers in the **Answers** section at the end (page 6).
4. **Pass mark: every question in the check correct.** If you missed one, rewatch that concept's video segment, reread the explanation, and redo only the questions you missed. Tick the concept off below, then move on.

It is strongly suggested that you **master each concept before starting the next one**. Each concept builds on the previous ones.

Concepts in this lesson

CONCEPT	VIDEO SEGMENT	QUESTIONS	MASTERED
1. A test you can run again	▶ 0:08-0:36	2	☐
2. The check decides the score	▶ 0:36-1:06	2	☐
3. Harder questions, and reading the answers	▶ 1:06-1:47	3	☐
4. Better checks, noise, and keeping the test	▶ 1:47-2:37	2	☐

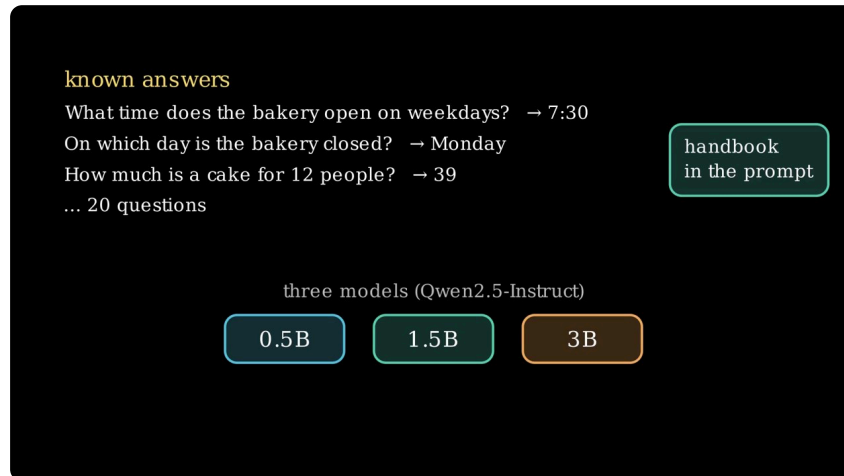
Before you start: episodes 3 (sampling), 5 (RAG) and 10 (quantization) of this series. Code: `code/p11_evaluation` (downloads three Qwen2.5 models, about 10 GB; about 15 minutes on a CPU).

CONCEPT 1 OF 4

A test you can run again

▶ Video 0:08–0:36

You changed a prompt, swapped a model, quantized it: is it better? *“It looked fine on a few tries”* is not an answer. You need an **evaluation**, a test you can run again and again. Start with questions that have **known answers**: here, 20 easy questions about the bakery handbook, with the handbook in the prompt (as in RAG), and three models: Qwen2.5 0.5B, 1.5B and 3B.



▶ 0:34 Questions with known answers, the handbook in the prompt, three models.

KEY IDEA

An evaluation = fixed questions + expected answers + an automatic check.

Check 1: A test you can run again

Answer all 2 questions before looking at the answers. Pass mark: all correct.

1.1 SHORT ANSWER

Why is “I tried three questions and it looked fine” a poor way to compare two prompts?

1.2 TRUE OR FALSE

“For a fair comparison, both models must get exactly the same questions and the same context.”

True ☐ False ☐

CONCEPT 2 OF 4

The check decides the score

▶ Video 0:36–1:06

The simplest check, **exact match**, gave the 1.5B model **1 / 20**. Its answers were right, just written as sentences: *“The bakery opens at 7:30 AM on weekdays.”* The test was wrong, not the model.

check 1 · exact match

1.5B: 1 / 20

expected: 7:30

The bakery opens at 7:30 AM on weekdays.

✗ marked wrong

the test was wrong, not the model

▶ 0:53 Exact match: 1 / 20, because full-sentence answers like “The bakery opens at 7:30 AM on weekdays.” fail.

check 2 · does the answer contain the fact?

0.5B	20 / 20
1.5B	20 / 20
3B	20 / 20

everyone passes: the test can't tell them apart

too easy

▶ 1:05 “Contains the fact”: 20 / 20 for every model. The easy test is saturated.

Checking whether the answer **contains** the expected fact gives **20 / 20** to all three models. That is a problem too: a test everyone passes cannot tell models apart. It is **too easy** (saturated).

KEY IDEA

A useful test has a fair check and room to separate good from better.

Check 2: The check decides the score

Answer all 2 questions before looking at the answers. Pass mark: all correct.

2.1 MULTIPLE CHOICE

Expected answer “monday”; the model says “The bakery is closed on Mondays.” Which check marks it correct?

- A. Exact match only
- B. Contains the expected word (lower-cased)
- C. Both
- D. Neither

2.2 SHORT ANSWER

All models score 20 / 20. What does that tell you, and what should you do?

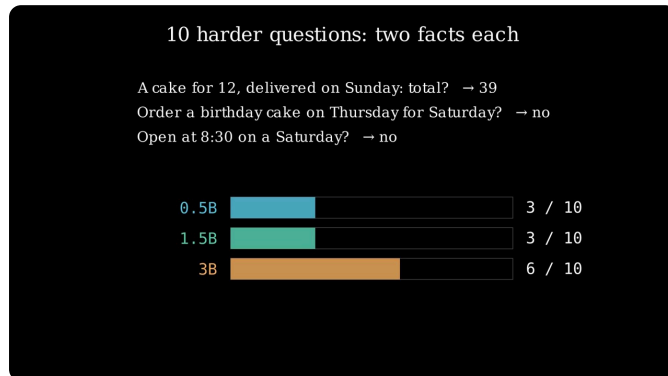
.....

CONCEPT 3 OF 4

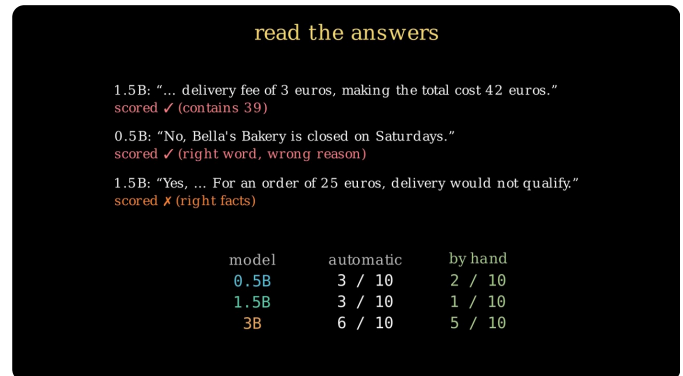
Harder questions, and reading the answers

▶ Video 1:06–1:47

Ten harder questions, each combining two facts (a cake for 12 delivered on Sunday: total? Can I order a birthday cake on Thursday for Saturday? Open at 8:30 on a Saturday?), separate the models: **3, 3 and 6** out of 10. The bigger model pulls ahead.



▶ 1:25 Ten harder questions: 3, 3 and 6 out of 10.



▶ 1:45 Reading the answers: automatic 3, 3, 6; by hand 2, 1, 5.

But **read the answers**. The 1.5B model said "... making the total cost 42 euros" and still scored, because "39" appears in its working. The 0.5B model said "No, ... closed on Saturdays": right word, wrong reason. The 1.5B model said "Yes, ... for an order of 25 euros, delivery would not qualify": right facts, marked wrong for starting with "yes". Checked by hand (final answer *and* its stated reason must be right), the real scores are **2, 1 and 5**: the 1.5B model's "No" for Monday 8:00, for example, came with the reason "it opens at 7:30 on weekdays", which contradicts its own answer.

KEY IDEA

An automatic score is only as good as its checker. Always read the failures, and a sample of the passes.

Check 3: Harder questions, and reading the answers

Answer all 3 questions before looking at the answers. Pass mark: all correct.

3.1 SHORT ANSWER

The first version of this test asked "What is the total for a cake for 8 people delivered on a Tuesday?" and expected 31 (28 + 3 euros delivery). What was wrong with the test itself?

3.2 MULTIPLE CHOICE

Which is a **false positive** of the automatic check?

- A. The 1.5B "42 euros" answer, scored correct because it contains 39
- B. The 1.5B "Yes, ... would not qualify" answer, scored wrong
- C. The 3B "31 euros" answer
- D. The 0.5B "No, closed on Mondays" answer

3.3 TRUE OR FALSE

"Automatic scores of 3, 3 and 6 prove the 3B model is exactly twice as good."

True ☐ False ☐

CONCEPT 4 OF 4

Better checks, noise, and keeping the test

▶ Video 1:47-2:37

Make checking easier: ask for a **structured answer** (a single number, or yes or no); use **several checks**; a stronger model can grade answers against a **rubric**, but check the grader too; and always **read a sample** by hand.

- ✓ ask for a structured answer: a number, yes or no
- ✓ use several checks
- ✓ a stronger model grades with a rubric (check the grader!)
- ✓ always read a sample by hand

▶ 2:00 Ways to make checking more reliable.

```
1 score = 0
2 for question, expected in TESTS:
3     answer = ask(model, question)
4     if check(answer, expected):
5         score += 1
6     else:
7         print("FAIL", question, "->", answer) # read these!
8 print(score, "/", len(TESTS))
```

▶ 2:21 An evaluation is a loop: ask, check, count, and print the failures.

Mind the **noise**: with 10 questions, one answer is 10%. With sampling, run each question several times; small differences on small test sets often mean nothing. Keep your test set and run it after **every change**: a new prompt, a new model, a quantized version. That is how you know it is better, and not just different.

KEY IDEA

Treat your test set like code tests: versioned, rerun on every change, and read when it fails.

Check 4: Better checks, noise, and keeping the test

Answer all 2 questions before looking at the answers. Pass mark: all correct.

4.1 CALCULATION

Model X scores 7 / 10 and model Y 6 / 10. How many questions is the difference, and why is it weak evidence?

4.2 HANDS-ON CODE

Try it yourself. In `code/p11_evaluation/evaluate.py`, change the system prompt to ask for structured answers, then rerun and compare the hard-set scores and outputs.

```
msg = [{"role": "system", "content": "Answer from the handbook below. Reply with only "
                                         "a number in euros, or only yes or no.\n\n" +
                                         HANDBOOK},
        {"role": "user", "content": question}]
```

ANSWERS

Answers and explanations

Mark each answer right or wrong. If a concept has any wrong answer, rewatch its segment (▶) and redo the questions you missed before moving on.

1. A test you can run again · rewatch ▶ 0:08-0:36

1.1 Too few, unchosen and unrepeatable examples: differences can be luck, and you cannot rerun it after the next change. A fixed test set gives comparable numbers every time.

1.2 True. Otherwise you are measuring the difference in inputs, not in models.

2. The check decides the score · rewatch ▶ 0:36-1:06

2.1 B. “mondays” contains “monday”; exact match compares the whole sentence.

2.2 The test is saturated and cannot rank the models; add harder questions. Scores at the ceiling hide real differences.

3. Harder questions, and reading the answers · rewatch ▶ 1:06-1:47

3.1 A cake for 8 costs 28 euros, below the 30-euro minimum for delivery: the correct answer is that it cannot be delivered. Test sets have bugs too. It was caught by reading the answers carefully, and replaced with a cake for 12 on a Tuesday ($39 + 3 = 42$ euros).

3.2 A. A false positive is a wrong answer counted as right. B is a false negative.

3.3 False. With 10 questions the scores are noisy, and the checker made errors (by hand: 2, 1, 5).

4. Better checks, noise, and keeping the test · rewatch ▶ 1:47-2:37

4.1 One question. A single answer can flip by chance (e.g. a near-tie, episode 10); you need more questions or repeated runs.

4.2 Your scores will change: answers become short (“39”, “no”), so the checks stop rewarding numbers that only appear in the reasoning and stop penalising “yes, but no”. Read the outputs to see whether the models also answer more or less correctly without room to reason. The answer format changes both the model's behaviour and what the checker can judge; that is why you rerun the whole test after any change.